

WEB PROGRAMLAMA

6. JavaScript Temelleri

Dr. Öğr. Üyesi Volkan KAYA

Erzincan Binali Yıldırım Üniversitesi Bilgisayar Mühendisliği

JavaScript

- Javascript, web sayfalarında dinamik içerik sağlamak ya da kullanıcıyla iletişim kurmak için kullanılan istemci taraflı bir dildir. Şu anda kullanılan bütün tarayıcılar tarafından desteklenmektedir.
 - İstemci taraflı dillerin amacı genellikle kullanıcının web sitesi ile etkileşimi sırasında sayfanın vereceği tepkileri tarayıcı üzerinden çabucak verebilmektir.
 - Örneğin kullanıcının şifresini yazması istenen bir web sayfasında, istemci taraflı bir dil ile bu durum kontrol edilebilir ve şifre girilmeden ilerlenmesi sunucuyla bağlantı kurulmaya gerek duyulmadan engellenebilir.
- En çok kullanılan istemci taraflı dil Javascript'tir.
 - Daha önceleri Visual Basic Script (VB Script) ile rekabet içinde olan Javascript, farklı teknoloji firmaları tarafından tercih edilmiş ve HTML5 standartları içerisinde tek başına yer alarak standart hale gelmiştir.
- Java programlama dili ile ilgisi yoktur. JavaScript kendi başına bir script dilidir. Yani gerçek bir programlama dilidir (HTML gibi sadece web sayfa düzenleme dili değildir).
- Javascript dili HTML etiketleri ve CSS kuralları ile oluşturulmuş web sayfalarına daha fazla işlevsellik katmak amacıyla kullanılır.
- Javascript HTML kodları arasına yerleştirilen küçük kodlardır.

JavaScript Nereye Yazılır?

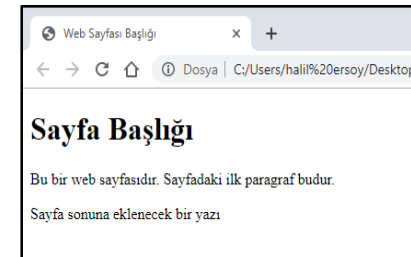
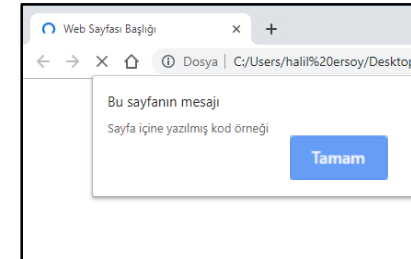
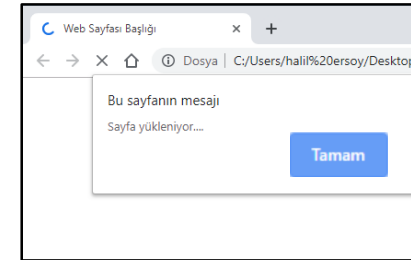
- HTML sayfalarına Javascript komutlarını yazmanın iki yeri vardır.
 - **1. Javascript Komutlarının HTML Sayfasına Açıkça Yazılması:** Javascript komutları HTML sayfasının içerisine açık biçimde yazılır.
 - a. Fonksiyon Kullanmadan
 - b. Fonksiyon Kullanarak
 - **2. Javascript Komutlarını Ayrı Dosyalar Şeklinde Yazmak:** Javascript komutları “js” uzantılı bir başka metin dosyası içerisine yazılır ve bu dosya HTML sayfasına dahil edilir.

JavaScript Nereye Yazılır?

(1.a. Javascript Komutlarının HTML Sayfasına Açıkça Yazılması - Fonksiyon Kullanmadan)

- HTML sayfalarında `<script>....</script>` etiketleri arasına Javascript komutları yazılır.

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Web Sayfası Başlığı</title>
    <script>
      alert('Sayfa yükleniyor....');
    </script>
  </head>
  <body>
    <h1>Sayfa Başlığı</h1>
    <p>Bu bir web sayfasıdır. Sayfadaki ilk paragraf budur.</p>
    <script>
      alert('Sayfa içine yazılmış kod örneği');
      document.write('Sayfa sonuna eklenecek bir yazı');
    </script>
  </body>
</html>
```

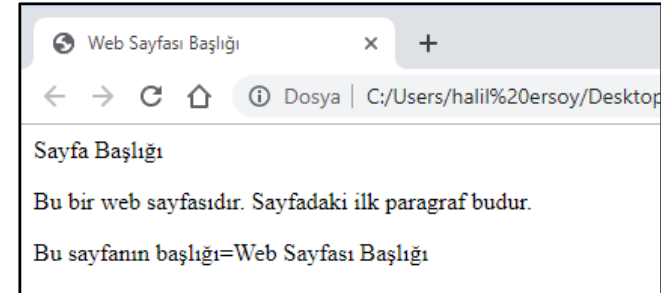


JavaScript Nereye Yazılır?

(1.b. Javascript Komutlarının HTML Sayfasına Açıkça Yazılması - Fonksiyon Kullanarak)

- Javascript komutlarının HTML sayfası içerisinde yazılma yönteminde sıklıkla kullanılan bir yol ise, komutları *Javascript fonksiyonları* halinde yazmak ve bu fonksiyonları istenilen yerden çağırarak çalıştırmaktır.

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Web Sayfası Başlığı</title>
    <script>
      function sayfaAdresiniGoster(){
        document.write("Bu sayfanın başlığı=" + document.title);
      }
    </script>
  </head>
  <body>
    <h1>Sayfa Başlığı</h1>
    <p>Bu bir web sayfasıdır. Sayfadaki ilk paragraf budur.</p>
    <script>
      sayfaAdresiniGoster();
    </script>
  </body>
</html>
```



JavaScript Nereye Yazılır?

(2. Javascript Komutlarını Ayrı Dosyalar Şeklinde Yazmak)

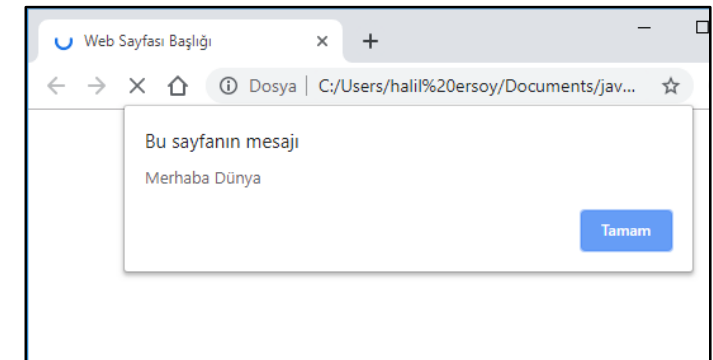
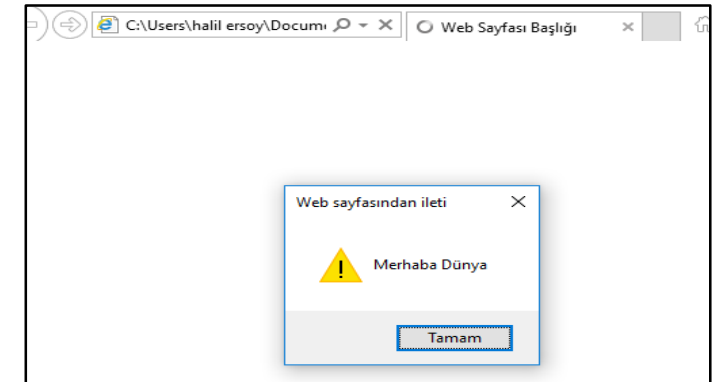
- Javascript komutları “js” uzantılı bir başka metin dosyası içerisine yazılır ve bu dosya HTML sayfasına dahil edilir.

scriptKomutlari.js

```
alert("Merhaba Dünya");
```

index.html

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Web Sayfası Başlığı</title>
  <script src="scriptKomutlari.js">
  </script>
</head>
<body>
  <h1>Sayfa Başlığı</h1>
  <p>Bu bir web sayfasıdır. Sayfadaki ilk paragraf budur.</p>
</body>
</html>
```



JavaScript Yazım Prensipleri

- Bir HTML sayfasına birden fazla Javascript dosyası eklenebilir.
- Bir `<script>` etiketiyle sadece bir Javascript dosyası eklenir. Birden fazla Javascript dosyası eklenmek istenirse, her biri için ayrı ayrı `<script>` etiketi yazılmalıdır.
- Aynı Javascript dosyası bir HTML sayfasına sadece bir kez dahil edilir.
- Javascript dosyalarının adlarında, tıpkı HTML dosya adlarında olduğu gibi noktalama işaretleri yer alamaz. Türkçe karakter ve boşluk karakterleri ise bazı tarayıcılarda çalışsa bile önerilmez.
- Javascript dosyasının adındaki harfler büyük ya da küçük olabilir. Ancak dosya adı ile “src” özelliğindeki dosya adındaki harflerin aynı şekilde olması gerekir.
- Javascript dosyalarının içinde ayrıca `<script>...</script>` etiketleri yazılmaz.
- Javascript dosyaları HTML dosyalarına dâhil edilirken, `<script>` etiketleri genellikle `<HEAD>...</HEAD>` etiketlerinin arasına yazılır. Bu çoğu zaman uygun bir yerdir ancak nadir durumlarda `<HEAD>` etiketleri sonrasında ve `<BODY>...</BODY>` etiketlerinin içinde de yer alabilir.
 - `<head>` etiketleri arasında
 - `<body>` etiketleri arasında

JavaScript'in Avantajları

- **Sunucuyla minimum etkileşim:** Bir internet sitesinin performansını yükseltmenin en iyi yolu sunucuyla iletişimi azaltmaktır. JavaScript bu noktada kullanıcı girdisini istemci tarafında (client-side) onaylayarak buna yardımcı olur. Talepleri sunucuya sadece ilk doğrulama kontrollerini çalıştırdıktan sonra gönderir. Bunun sonucunda kaynak kullanımı ve sunucuya gönderilen talep miktarı önemli ölçüde azaltılır.
- **Zengin, kullanıcı dostu arayüzler:** JavaScript kullanarak istemci tarafında interaktif arayüzler oluşturulabilir. Örneğin slayt gösterileri, kaydırma butonları, roll-over efektleri, sürükle & bırak özelliği ve bunun gibi diğer şeyler.
- **Kullanıcıya anında geri dönüş:** JavaScript kullanarak kullanıcıların anında yanıt almaları sağlanabilir. Örnek olarak form dolduran bir kullanıcı alanlardan bir tanesini boş bırakırsa, JavaScript doğrulaması olmadığında, kullanıcının bir alanı boş bıraktığını anlayabilmesi için sayfanın tekrar yüklenmesini beklemesi gerekir. Ancak JavaScript ile anında uyarılır.
- **Kolay hata ayıklama:** JavaScript yorumlanan (interpreted) bir dildir. Bunun anlamı, yazılan kodun her bir satırın şifre çözülür gibi çözüldüğüdür. Herhangi bir hata ortaya çıktığında, sorunun tam olarak hangi satır numarasında olduğu görülebilir.

JavaScript Yazma, Çalıştırma, Test Etme

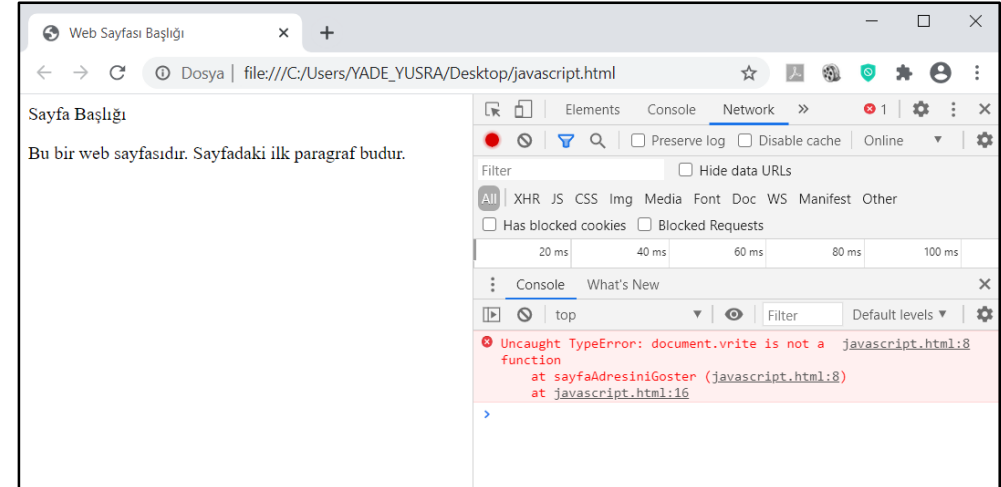
- Javascript komutları tıpkı HTML etiketleri gibi metin tabanlı yazılır ve tarayıcı üzerinde derlenip çalıştırılır. Bu nedenle HTML komutları hangi program ya da editörde yazılıyorsa, aynı program veya editörde Javascript komutları da yazılabilir.
- HTML, CSS ve Javascript komutlarını yazmak için yardımcı editörler;
 - Adobe Dreamweaver® (Ücretli)
 - Microsoft Visual Studio® (Ücretli)
 - Brackets® (Ücretsiz)
 - **Notepad++® (Ücretsiz)**
 - Visual Studio Code® (Ücretsiz)
- Yukarıdaki editörler HTML sayfalarını etiketler, CSS kuralları ve Javascript komutları ile yazma aşamasında yardımcı olurken, yazılan kodların çıktısını göstermemekte veya eksik göstermektedir. Bu nedenle hangi editör kullanılıyorsa kullanılsın, HTML sayfalarının ve içerisinde Javascript komutlarının çıktısını görmek için gerçek tarayıcıların kullanılması tavsiye edilir.
 - Günümüzde popüler olan tüm tarayıcılar (Google Chrome®, Microsoft Edge®, Microsoft Internet Explorer®, Safari®, Opera®, Firefox® vb.) Javascript'i derleyip çalıştırabilmektedir. Hatta Javascript komutlarında olası bir hatayı kendi içerisinde ayrı bir pencerede gösterebilmektedir.

JavaScript Hatalı Kodlarının Tarayıcıda İzlenmesi

- Örnek: Aşağıdaki kodlarda Script bloku içinde document.write() kodu yerine hatalı biçimde document.vrite() komutu kullanılmıştır.

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Web Sayfası Başlığı</title>
    <script>
      function sayfaAdresiniGoster(){
        document.vrite("Bu sayfanın başlığı=" + document.title);
      }
    </script>
  </head>
  <body>
    <h1>Sayfa Başlığı</h1>
    <p>Bu bir web sayfasıdır. Sayfadaki ilk paragraf budur.</p>
    <script>
      sayfaAdresiniGoster();
    </script>
  </body>
</html>
```

- Javascript kodlarındaki yazım (sözdizim) hataları HTML etiketlerinin çalışmasını engellemez. Ancak Javascript komutlarının çalışması, hatalı yerden itibaren durur ve devamındaki Javascript komutları çalıştırılmaz. Örnekteki çıktıda Javascript komutları ile yazılması gereken yazının ekrana gelmediğini fark edebilirsiniz.
- Hatanın hangi satır ve neden olduğunu anlamak için tarayıcıların “geliştirici seçenekleri” (developer options) açılmalıdır. Örnekte kullanılan Google Chrome® için bu seçeneği klavyeden F12 tuşuna basarak açabilirsiniz. Bu seçeneği açtığınızda tarayıcının alt kısmında sayfanın kodlarıyla ilgili bilgiler alabileceğiniz pencereler açılır.



JavaScript Hangi İşler İçin Kullanılır?

- **Tarayıcı Algılama:** Bir HTML sayfasının başında yazılacak Javascript komutları ile tarayıcının adı ve farklı özellikleri öğrenilebilir, ardından kullanıcıya kendi tarayıcısı ile uyumlu içerik veya sayfalar gösterilebilir.
- **Bilgi Girişlerinde Eksik ya da Hatalı Girişleri Kontrol Etme:** Javascript ile içinde bulunduğunuz bir sayfadaki herhangi bir form alanına girilen eksik bilgi tespit edilebilir ve kullanıcıya derhal uyarı mesajı verilebilir.
- **Kullanıcı Hareketleri veya Tercihlerine Göre Farklı İçerik Sunma:** Javascript ile kullanıcıların farklı içerik öğeleri ile etkileşimi (tıklama, seçme, iptal etme vb.) algılanabilir ve sonrasındaki uygun içerik, sayfaya yansıtılabilir.
- **Sayfa İçeriğinde Belirli Sürelerde Güncelleme Yapma:** Javascript ile kullanıcılara anlık değişimlerin fazla olduğu konularda, sürekli yeni/güncel bilgiler sunulabilir.
- **Matematiksel Hesaplamalar Yapma:** Javascript ile kullanıcıların sayısal veya sözel veri girmesi ve bu verilerin belirli formüllerde hesaplanarak sonuçların ekrana yazılması sağlanabilir.

Yapılabilecek bu işlerde Javascript haricinde yollar da kullanılabilir, ancak Javascript hem hızlı hem de sunucuyu yormayan bir çözümdür.

JavaScript Dilinin Genel Özellikleri

- İsimlendirme Kuralları

- Değişkenler, fonksiyon isimlendirmeleri ve komutlarda büyük ve küçük harf ayrımı vardır. Örneğin alert() komutu doğru, Alert() ifadesi ise geçersizdir.
- Değişken, fonksiyon ve diğer isimlendirmelerde İngiliz alfabesindeki harfler (A-Z / a-z), rakamlar ve alt tire (_) ve dolar (\$) sembolleri kullanılabilir. Bunların dışında sembol kullanılmasına izin verilmez.
- Değişken, fonksiyon ve diğer isimlendirmelerde ilk karakter rakam olamaz, mutlaka harf veya alt tire karakteri olmalıdır.
- \$ sembolünün ilk karakter olarak kullanılması önerilmez, bu Javascript kütüphaneleri ile birlikte kullanıldığında çakışmalara neden olabilir.
- Javascript diline has komut ve ifadeler, isimlendirmede kullanılamazlar.
- Kullanılan isimdeki karakter sayısı sınırsızdır.
- Aynı isimde iki değişken, fonksiyon ya da nesne aynı web sayfası tarafından aynı anda tanımlanamaz. Ancak fonksiyonların kendi içindeki lokal değişkenler, bir başka fonksiyon içinde yine aynı isimle lokal olarak tanımlanabilirler.

```
<script>
    var birSayi=11;
    if((birSayi%2) == 0)
    {
        alert(birSayi+" sayisi çift sayıdır.");
    }
    else
    {
        alert(birSayi+" sayisi tek sayıdır.");
    }
</script>
```

- Parantezler

- Javascript'teki bazı komutların yapısı gereği, ilgili bazı ifadelerin veya komutların gruplanması gerekebilir. Bu grupları oluşturmak için normal parantezler () ya da küme parantezleri { } kullanılır.

- Komut Satırları ve Noktalı Virgöl

- Popüler birçok dile benzer biçimde Javascript'te komutlar noktalı virgül karakteri ile biter, ardından bir başka komut başlar.

```
<script>
    var a=5; var b=4; document.write("toplam= " + (a + b));
</script>
```

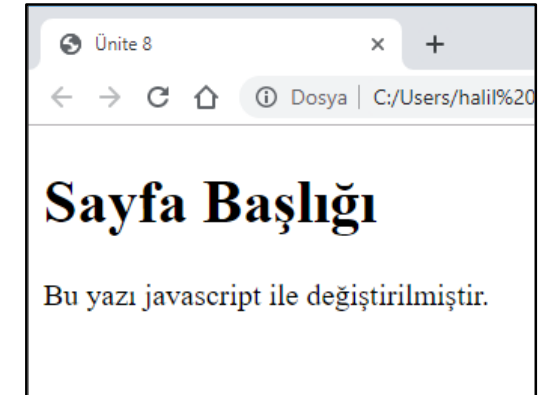
- Nesne Tabanlı Programlama Olanağı

- Javascript, nesne tabanlı programlama dilidir. Hem yeni sınıflar ve nesneler yaratmaya, hem de HTML etiketleri ile oluşturulmuş HTML belgesinin nesnelerine ulaşarak kodlama yapmayı sağlar.

JavaScript ile HTML Etiketlerine Erişmek

- Javascript dili ile HTML sayfalarındaki tüm etiketlerin özelliğini veya içeriği değiştirmek mümkündür. Bunun için document.getElementById() komutu kullanılır.

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>HTML Etiketlerine Erişmek</title>
  </head>
  <body>
    <h1>Sayfa Başlığı</h1>
    <p id="yazi1">Bu bir paragraftır.</p>
    <script>
      document.getElementById("yazi1").innerText="Bu yazı javascript ile değiştirilmiştir.";
    </script>
  </body>
</html>
```

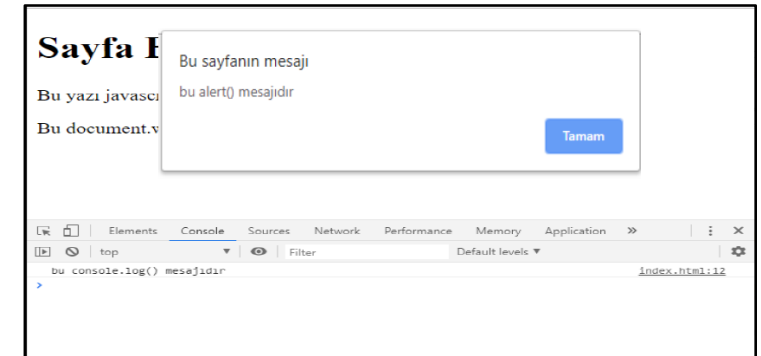


- document.getElementById() komutunun doğru şekilde çalışması için aşağıdaki koşulların sağlanmış olması gerekir;
 - HTML etiketlerine Javascript komutları ile erişmek için "id" özelliği ile benzersiz bir isim verilmelidir. Aynı HTML sayfası içerisinde aynı id'ye sahip başka eleman olmamalıdır.
 - document.getElementById() fonksiyonunun parantezleri içerisine bu "id" bilgisi çift tırnak/tek tırnak içerisinde yazılmalıdır.
 - document.getElementById() komutu, etiket yüklendikten sonra çalışabilir, öncesinde çalışmaz. (Yani <p id="yazi1">Bu bir paragraftır.</p> bu satırı script den sonra yazarsak hata oluşur.)

JavaScript ile Basit Mesajlar Yazdırma

- Mesaj yazdırma komutları;
 - alert();
 - document.write();
 - console.log();

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>JavaScript ile Mesaj</title>
  </head>
  <body>
    <h1>Sayfa Başlığı</h1>
    <p id="yazi1">Bu bir paragraftır.</p>
    <script>
      document.getElementById("yazi1").innerText="Bu yazı javascript ile değiştirilmiştir";
      console.log("bu console.log() mesajıdır");
      document.write("Bu document.write() mesajıdır");
      alert("bu alert() mesajıdır");
    </script>
  </body>
</html>
```



JavaScript'de Değişkenler

- Değişken tanımlamak için *var* komutu kullanılır.

```
<script>
  var a;
  var b=50;
  var c=3.14;
  var d="merhaba";
  var e="dünya";
  f=80; //var kullanılmadığında Global
  document.write("a=" + a + "</br>");
  document.write("b=" + b + "</br>");
  document.write("c=" + c + "</br>");
  document.write("d=" + d + "</br>");
  document.write("e=" + e + "</br>");
  document.write("f=" + f + "</br>");
</script>
```

```
a=undefined
b=50
c=3.14
d=merhaba
e=dünya
f=80
```


JavaScript’de Değişken Tanımlama Kuralları

Kural	Örnek ve Çıktı
1. Değişkenler var komutu ile tanımlanır, istenirse tanımlama sırasında ilk değer verilebilir.	<pre>var b; b=50; YA DA var b=50;</pre>
2. İlk değeri verilmeyen değişkenlerin değeri “undefined” olur.	<pre>var a; document.write(a); → undefined</pre>
3. Aynı var ifadesi ile birden çok değişken tanımlanabilir. Yandaki örnekte sadece c değişkeninin değeri 30 olur, diğerlerinin “undefined” olur.	<pre>var a,b,c=30; document.write(a); → undefined document.write(b); → undefined document.write(c); → 30</pre>
4. Var komutu ile aynı isimde bir başka değişken tekrar tanımlanabilir, bu durumda önceki değişken yok edilmiş olur.	<pre>var a=20; document.write(a); → 20 var a = 75; document.write(a); → 75</pre>
5. Aynı satırda birden fazla değişkene aynı ilk değer verilebilir.	<pre>var a=b=c=40; document.write(a); → 40 document.write(b); → 40 document.write(c); → 40</pre>
6. Bir <script> blokunda tanımlanan değişken, bu blok kapandıktan sonra açılacak bir sonraki script blokunda da kullanılabilir.	<pre><script> var a=30; </script> <p>yazı</p> <script> document.write(a); → 30 </script></pre>

JavaScript'de Veri Türleri

- Değişkenlerin içerisinde saklanacak veri çeşidine göre *türü* vardır.
- Değişkene atanacak *ilk değer*in türüne göre, değişken türü belirlenmiş olur.

```
<script>
  var a; //tür: belirsiz (undefined)
  a=50; //tür: sayı (number)
  var b=3.14; //tür: sayı (number)
  var c="merhaba"; //tür: metin (string)
  var d=123e5; //tür: bilimsel sayı (123 x 105) (number)
  var e='t'; //tür: metin (string)
</script>
```

JavaScript'de Tür Dönüşümleri

- Değişkenlere farklı türde değer vererek o değişkenin *türünü değiştirmiş* oluruz.
- `typeof()` komutu değişkenin türünü elde etmemize yarar.

```
var a=30;  
document.write(typeof(a)); → number  
a=3.14;  
document.write(typeof(a)); → number  
a=123e5;  
document.write(typeof(a)); → number  
  
a = "merhaba";  
document.write(typeof(a)); → string  
a='r';  
document.write(typeof(a)); → string  
  
var a= 4;  
var b= 8;  
var c= b > a;  
document.write(typeof(c)); → boolean  
  
var a=[10,20,30,40,50];  
document.write(typeof(a)); → object
```

JavaScript'de Operatörler

- Değişkenlerin değerlerini değiştirmeye veya değerlerini kontrol etmeye yarayan komutlara operatör denir.
- Operatörler yaptıkları işler bakımından 3 sınıfa ayrılırlar;
 - Aritmetik Operatörler:** +, -, *, /, %, ++, --

```
var a=5, b=1;  
var sonuc;  
sonuc = a + b++;  
document.write("sonuc=" + sonuc + "<br>");  
document.write("a=" + a + "<br>");  
document.write("b=" + b + "<br>");
```

```
sonuc=6  
a=5  
b=2
```

- Karşılaştırma Operatörleri:** ==, !=, === (Değerleri ve türleri eşit), !==, >, <, >=, <=

```
var a = 10  
a === "10"
```

```
false
```

- Mantıksal Operatörler:** &&, ||, !

```
var a=10, b=20;  
a <= 10 && b < 100
```

```
true
```

Kaynaklar

- İnternet Programcılığı 1 (Ebubekir YAŞAR - Turgut ÖZSEVER, Ekin Yayınevi, 5. Baskı, 2017)
- İnternet Programcılığı 2 (Ebubekir YAŞAR - Turgut ÖZSEVER, Ekin Yayınevi, 5. Baskı, 2016)
- İnternet Programcılığı II (Atatürk Üniversitesi Açıköğretim Fakültesi Yayını, 2020)
- İnternet Kaynakları